AWS Connect — Custom IAM Policies (replace both AWS-managed policies)

> Audience: platform / SRE engineers and the customer's AWS admin. Goal: run the **entire** Digitomics AWS connector — billing/FinOps sync, inventory dashboards, FOCUS ingest, commitment analytics, anomaly detection, and the read side of the DevOps chat agent — on tightly-scoped customer-managed policies only, dropping **both** `ReadOnlyAccess` **and** `AWSBillingReadOnlyAccess`, and never granting a permission that can read secret material or mutate the account.

This version replaces the earlier "Core / split / full" layout, which repeated the same statements three times. There are now **six policies**, and **no IAM action appears in more than one of them.**

---

1. TL;DR — what to attach

#	Policy	Required?	Replaces
1	`DigitomicsAwsConnectBillingRead`	**Yes**	`AWSBillingReadOnlyAcces`
2	`DigitomicsAwsConnectInventoryRead`	**Yes**	the inventory half of `ReadOnlyAccess`
3	`DigitomicsAwsConnectDevOpsChatRead`	Optional	rest of `ReadOnlyAccess` (only if you use the DevOps chat agent for open-ended reads)
4	`DigitomicsAwsConnectFocusExportWrite`	Optional	— (enables automatic FOCUS 1.2 export provisioning)
5	`DigitomicsAwsConnectAuditRead`	Optional	— (CloudTrail context on cost anomalies)
6	`DigitomicsAwsConnectCommitmentExecution`	Opt-in, **not** recommended	— (lets Digitomics *buy* Savings Plans)

Minimum working set: #1 + #2. That runs every dashboard tile, FinOps sync, FOCUS ingest, commitment analytics, and anomaly feed. Attach #3–#6 only for the matching feature. Do **not** attach `ReadOnlyAccess` or `AWSBillingReadOnlyAccess` alongside these.

> **Account-level prerequisite (unchanged).** `ce:*` returns `AccessDenied` under **any** policy — custom or managed — until the payer account enables **Billing → IAM user and role access to billing information → Activate IAM Access** and Cost Explorer has been turned on once. See `docs/AWS_CONNECTION.md` §0.

---

2. How the connector authenticates (context for the policies)

Saving a connection (`backend/Tools/connections/connections_routes.py` → `validate_aws_credentials` in `backend/services/aws_credentials.py`) makes one `sts:GetCallerIdentity` call; nothing is persisted if STS rejects it. After that, every feature runs as the **same** IAM principal:

Path	Entry point
Connect wizard probe	`connections_routes.py:1416`
Billing + inventory sync	`services/billing_sync_service.py:370` → `AwsDashboardService.build_dashboard`
FOCUS line-item ingest (billing-sync tail hook)	`services/focus/ingest.py` → `run_focus_ingestion_for_tenant`
Commitment inventory / pricing sync	`services/commitment/collector.py`, `pricing.py`
Cost-intelligence anomaly fusion / events	`services/cost_intelligence/fusion.py`, `events.py`
FinOps chat agent	`agents/finops_agent.py` → `agents/tools/aws_tools.py`
DevOps chat agent (read)	`agents/devops_agent.py` → `agents/tools/Devops_aws_tools/…` (AWS API MCP)

The DevOps agent can also issue mutating calls, but those are gated in-app by the HITL layer (`agents/tools/Devops_aws_tools/policies/aws_policy.py`) **and** are simply not granted here. Write access for that agent is a separate, deliberately scoped policy — see `docs/AWS_CONNECTION.md` "Pack C".

---

3. Design principles

1. **Only what the code calls** — or a direct read-only sibling (pagination variant, tag read). §5 is the audit table. Uniformly-safe read surfaces use a `service:Describe*` / `service:List*` wildcard matching the AWS-managed `Amazon<Service>ReadOnlyAccess` policy.
2. **No secret material.** Services whose read surface includes sensitive getters are enumerated,

not wildcarded:
    - `lambda` — `ListFunctions` only (never `GetFunction*`; note `ListFunctions` itself returns env-var values by design — keep secrets in Secrets Manager / SSM SecureString referenced by ARN).
    - `s3` — bucket list + location; object reads scoped to the `digitomics-focus-export-*` buckets the connector creates.
    - `dynamodb` — table metadata only, never `GetItem` / `Query` / `Scan`.
    - `iam` — three account-summary calls, never policies / keys / MFA.
    - `ssm` — `GetParameter` scoped to the public `/aws/service/*` namespace.
    - `cloudfront` — `ListDistributions` only, never `GetDistribution*`.
3. **No mutation** except the opt-in, name-scoped policy #4 (`digitomics-focus-export-*` + BCM Data Exports) and the explicitly-opt-in policy #6.
4. **No overlap.** Every action string lives in exactly one of the six policies.
5. **Graceful degradation is built in.** Every collector wraps its AWS call in `try/except (BotoCoreError, ClientError)` and logs + continues. A missing optional permission narrows a tile; it never breaks the connection.

---

4. The six policies

4.1 `DigitomicsAwsConnectBillingRead` — required (replaces `AWSBillingReadOnlyAccess`)

Everything the codebase reads on the billing / cost-management surface. The managed `AWSBillingReadOnlyAccess` also carries `invoicing:*`, `tax:*`, `payments:*`, `freetier:*`, `account:*`, `cur:*`, `consolidatedbilling:*` and the console `billing:*` namespace — **no Digitomics code path calls any of them** (§6).

`Sid` — grant	Code	
`AccountIdentity` — `sts:GetCallerIdentity`	`aws_credentials.py:384`, `aws_dashboard_service.py:598`, `focus/ingest.py:191`	validate t 12-digit a billing sn
`CostExplorerRead` — `ce:Get*`, `ce:Describe*`, `ce:List*`	`aws_dashboard_service.py` (`_get_daily_cost_trend`, `_get_cost_by_service`, `_get_total_cost`, `_get_forecast`, `_get_anomalies`, `_get_rightsizing_recommendations`, `_get_commitment_coverage`), `aws_tools.py` (`aws_get_cost_and_usage`, `… with_resources`, `… forecast`, …	daily trer spend, m cost anor Plans cov purchase FOCUS C anomaly **statemer**

	_categories, …_anomalies, …_reservation_recommendations), focus/ingest.py:230, cost_intelligence/fusion.py:300	**hard-fail** (aws_das ce:* ha member Get* / De
BudgetsRead — budgets:ViewBudget, budgets:Describe*, budgets:ListTagsForResource	aws_tools.py:496 (describe_budgets), services/budget_threshold_service.py	"show m budget-t ingest. M ModifyBu budget *a*
SavingsPlansRead — savingsplans:DescribeSavingsPlans, DescribeSavingsPlansOfferings, DescribeSavingsPlanRates, DescribeSavingsPlansOfferingRates, ListTagsForResource	commitment/collector.py:67, commitment/executor.py:50, commitment/pricing.py:32, commitment/verify.py:25	active Sa offering/ simulatio verificati savingsp **excluded**
PriceListRead — pricing:GetProducts, pricing:DescribeServices, pricing:GetAttributeValues, pricing:ListPriceLists, pricing:GetPriceListFileUrl	aws_tools.py:_run_pricing_mcp_estimate (AWS Pricing MCP)	"estimate deploy it mutating
FocusDataExportRead — bcm-data-exports:ListExports, GetExport, ListTables, GetTable, ListTagsForResource	focus/aws_export_provisioner.py:150	detect ar Export be data
FocusExportBucketRead — s3:ListBucket, s3:GetObject on digitomics-focus-export-*	focus/ingest.py:379/393	read the files AWS (resource bucket)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AccountIdentity",
      "Effect": "Allow",
      "Action": "sts:GetCallerIdentity",
      "Resource": "*"
    },
    {
      "Sid": "CostExplorerRead",
```

```
      "Effect": "Allow",
      "Action": [
        "ce:Get*",
        "ce:Describe*",
        "ce:List*"
      ],
      "Resource": "*"
    },
    {
      "Sid": "BudgetsRead",
      "Effect": "Allow",
      "Action": [
        "budgets:ViewBudget",
        "budgets:Describe*",
        "budgets:ListTagsForResource"
      ],
      "Resource": "*"
    },
    {
      "Sid": "SavingsPlansRead",
      "Effect": "Allow",
      "Action": [
        "savingsplans:DescribeSavingsPlans",
        "savingsplans:DescribeSavingsPlansOfferings",
        "savingsplans:DescribeSavingsPlanRates",
        "savingsplans:DescribeSavingsPlansOfferingRates",
        "savingsplans:ListTagsForResource"
      ],
      "Resource": "*"
    },
    {
      "Sid": "PriceListRead",
      "Effect": "Allow",
      "Action": [
        "pricing:GetProducts",
        "pricing:DescribeServices",
        "pricing:GetAttributeValues",
        "pricing:ListPriceLists",
        "pricing:GetPriceListFileUrl"
      ],
      "Resource": "*"
    },
    {
      "Sid": "FocusDataExportRead",
      "Effect": "Allow",
      "Action": [
        "bcm-data-exports:ListExports",
        "bcm-data-exports:GetExport",
        "bcm-data-exports:ListTables",
        "bcm-data-exports:GetTable",
        "bcm-data-exports:ListTagsForResource"
      ],
      "Resource": "*"
```

```
    },
    {
      "Sid": "FocusExportBucketRead",
      "Effect": "Allow",
      "Action": [
        "s3:ListBucket",
        "s3:GetObject"
      ],
      "Resource": [
        "arn:aws:s3:::digitomics-focus-export-*",
        "arn:aws:s3:::digitomics-focus-export-*/*"
      ]
    }
  ]
}
```

4.2 `DigitomicsAwsConnectInventoryRead` — required

The resource-inventory, waste-scan, utilization and AMI-resolution grants — everything the code reads that is **not** billing.

`Sid` — grant	Code	
`ComputeAndNetworkInventory` — `ec2:Describe*`, `elasticloadbalancing:Describe*`	`aws_credentials.py:216` (`DescribeRegions`), `aws_dashboard_service.py` (`DescribeInstances`, `DescribeVolumes`, `DescribeAddresses`, `DescribeSnapshots`, `DescribeVpcs`, `DescribeNatGateways`, `DescribeLoadBalancers`), `commitment/collector.py:94` (`DescribeReservedInstances`), `aws_tools.py:411` (`DescribeImages`)	regior runnir unatta Elastic snaps count Instan AMI r `ec2:D` exact `Amazc` no da mutat
`DatabaseAndCacheInventory` — `rds:Describe*`, `rds:ListTagsForResource`, `elasticache:Describe*`, `redshift:Describe*`, `dynamodb:ListTables`, `dynamodb:DescribeTable`, `dynamodb:ListTagsOfResource`	`aws_dashboard_service.py` (`DescribeDBInstances`, `DescribeCacheClusters`, `DescribeClusters`, `ListTables`), `commitment/collector.py:128` (`DescribeReservedDBInstances`)	RDS / Redsh invent RDS R Dynar (no `G` `Scan`
`ContainerInventory` —		

`ecs:Describe*`, `ecs:List*`, `eks:Describe*`, `eks:List*`	`aws_dashboard_service.py:1488/1514/1926/1942`	ECS cl count
`ServerlessInventory` — `lambda:ListFunctions`	`aws_dashboard_service.py:1122/1535`	functi Lamb `GetFu` grante
`StorageInventory` — `s3:ListAllMyBuckets`, `s3:GetBucketLocation`	`aws_dashboard_service.py:1775/1785`	bucke per-b
`ContentDeliveryInventory` — `cloudfront:ListDistributions`	`aws_dashboard_service.py:1750`	distrib enabl
`ObservabilityRead` — `cloudwatch:DescribeAlarms`, `cloudwatch:ListMetrics`, `cloudwatch:GetMetricStatistics`, `cloudwatch:GetMetricData`	`aws_dashboard_service.py:1078/1128/1157/1710`	alarm CPU (invoca Lamb (idle F
`IamAccountMetadataRead` — `iam:ListUsers`, `iam:ListAccountAliases`, `iam:GetAccountSummary`	`aws_dashboard_service.py:1805`	IAM-u IAM t summ reads passw
`PublicSsmAmiParameterRead` — `ssm:GetParameter`, `ssm:GetParameters` on `/aws/service/*`	`aws_tools.py:424`	resolv Amaz for th Resou AWS- param

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ComputeAndNetworkInventory",
      "Effect": "Allow",
      "Action": [
        "ec2:Describe*",
        "elasticloadbalancing:Describe*"
      ],
      "Resource": "*"
    },
    {
      "Sid": "DatabaseAndCacheInventory"
```

```
    "Sid": "DatabaseAndCacheInventory",
    "Effect": "Allow",
    "Action": [
      "rds:Describe*",
      "rds:ListTagsForResource",
      "elasticache:Describe*",
      "redshift:Describe*",
      "dynamodb:ListTables",
      "dynamodb:DescribeTable",
      "dynamodb:ListTagsOfResource"
    ],
    "Resource": "*"
  },
  {
    "Sid": "ContainerInventory",
    "Effect": "Allow",
    "Action": [
      "ecs:Describe*",
      "ecs:List*",
      "eks:Describe*",
      "eks:List*"
    ],
    "Resource": "*"
  },
  {
    "Sid": "ServerlessInventory",
    "Effect": "Allow",
    "Action": "lambda:ListFunctions",
    "Resource": "*"
  },
  {
    "Sid": "StorageInventory",
    "Effect": "Allow",
    "Action": [
      "s3:ListAllMyBuckets",
      "s3:GetBucketLocation"
    ],
    "Resource": "*"
  },
  {
    "Sid": "ContentDeliveryInventory",
    "Effect": "Allow",
    "Action": "cloudfront:ListDistributions",
    "Resource": "*"
  },
  {
    "Sid": "ObservabilityRead",
    "Effect": "Allow",
    "Action": [
      "cloudwatch:DescribeAlarms",
      "cloudwatch:ListMetrics",
      "cloudwatch:GetMetricStatistics",
      "cloudwatch:GetMetricData"
    ],
```

```
      ],
      "Resource": "*"
    },
    {
      "Sid": "IamAccountMetadataRead",
      "Effect": "Allow",
      "Action": [
        "iam:ListUsers",
        "iam:ListAccountAliases",
        "iam:GetAccountSummary"
      ],
      "Resource": "*"
    },
    {
      "Sid": "PublicSsmAmiParameterRead",
      "Effect": "Allow",
      "Action": [
        "ssm:GetParameter",
        "ssm:GetParameters"
      ],
      "Resource": [
        "arn:aws:ssm:*::parameter/aws/service/*",
        "arn:aws:ssm:*:*:parameter/aws/service/*"
      ]
    }
  ]
}
```

4.3 `DigitomicsAwsConnectDevOpsChatRead` — optional

Policies #1 + #2 cover every service the **automated** collectors touch. The DevOps chat agent (`agents/tools/Devops_aws_tools`) can be asked to *read* any service. Attach this so it can answer "list my CloudFormation stacks / SNS topics / Route 53 zones / log groups / …" without re-introducing full `ReadOnlyAccess`. `Describe*` / `List*` plus a few safe `Get*`; no data-plane or secret getter.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DevOpsChatBroadRead",
      "Effect": "Allow",
      "Action": [
        "autoscaling:Describe*",
        "cloudformation:Describe*",
        "cloudformation:List*",
        "logs:Describe*",
        "logs:ListTagsForResource",
        "sns:List*",
        "sns:GetTopicAttributes",
        "sns:GetSubscriptionAttributes",
```

```
        "sqs:ListQueues",
        "sqs:ListQueueTags",
        "sqs:GetQueueAttributes",
        "ecr:Describe*",
        "ecr:List*",
        "route53:List*",
        "route53:Get*",
        "elasticbeanstalk:Describe*",
        "states:List*",
        "states:Describe*",
        "events:List*",
        "events:Describe*",
        "efs:Describe*",
        "kms:List*",
        "kms:DescribeKey",
        "acm:List*",
        "acm:DescribeCertificate",
        "secretsmanager:ListSecrets",
        "ssm:DescribeParameters",
        "tag:GetResources",
        "tag:GetTagKeys",
        "servicequotas:ListServiceQuotas",
        "servicequotas:GetServiceQuota",
        "organizations:Describe*",
        "organizations:List*"
      ],
      "Resource": "*"
    }
  ]
}
```

> `secretsmanager:ListSecrets` returns names/metadata only, never `SecretString`. `kms:DescribeKey` returns key metadata, not key material. `logs:Describe*` lists log groups/streams; `logs:GetLogEvents` / `logs:FilterLogEvents` (log **content**) are intentionally omitted.

4.4 `DigitomicsAwsConnectFocusExportWrite` — optional (write, resource-scoped)

Lets `services/focus/aws_export_provisioner.py` create — once, in *your* account — a private encrypted S3 bucket `digitomics-focus-export-<account_id>` and a native **FOCUS 1.2** BCM Data Export, so FOCUS ingest gets resource/SKU/invoice/ commitment-level detail instead of the Cost-Explorer-only rollup. Without it the ingester records `denied` once and never retries; Cost Explorer data is unaffected (`docs/AWS_FOCUS_SETUP.md`).

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
```

```
      "Sid": "FocusExportBucketWrite",
      "Effect": "Allow",
      "Action": [
        "s3:CreateBucket",
        "s3:PutBucketPublicAccessBlock",
        "s3:PutEncryptionConfiguration",
        "s3:PutBucketPolicy",
        "s3:GetBucketPolicy"
      ],
      "Resource": "arn:aws:s3:::digitomics-focus-export-*"
    },
    {
      "Sid": "FocusDataExportWrite",
      "Effect": "Allow",
      "Action": [
        "bcm-data-exports:CreateExport",
        "bcm-data-exports:UpdateExport",
        "bcm-data-exports:TagResource"
      ],
      "Resource": "*"
    }
  ]
}
```

4.5 `DigitomicsAwsConnectAuditRead` — optional

`services/cost_intelligence/events.py:306` calls `cloudtrail.lookup_events` to attach "who changed what, when" context to a detected cost anomaly — 90-day management-event history, audit metadata not secrets, opt-in because some orgs treat the trail as sensitive.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "CostAnomalyRootCauseAudit",
      "Effect": "Allow",
      "Action": [
        "cloudtrail:LookupEvents",
        "cloudtrail:DescribeTrails",
        "cloudtrail:GetTrailStatus"
      ],
      "Resource": "*"
    }
  ]
}
```

4.6 `DigitomicsAwsConnectCommitmentExecution` — opt-in, not recommended by default

`savingsplans:CreateSavingsPlan` (`commitment/executor.py:69`) actually buys a Savings Plan — a multi-year financial commitment. Gated in-app by `settings.commitment_execution_enabled` + HITL approval. Attach **only** if you have set `COMMITMENT_EXECUTION_ENABLED=true` and want Digitomics to place live purchases.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "CommitmentPurchaseExecutionOptIn",
      "Effect": "Allow",
      "Action": "savingsplans:CreateSavingsPlan",
      "Resource": "*"
    }
  ]
}
```

---

5. Full audit table — every AWS call in the repo → policy

boto3 call	IAM action
`sts.get_caller_identity`	`sts:GetCallerIdentity`
`sts.assume_role`	`sts:AssumeRole`
`ce.get_cost_and_usage`	`ce:GetCostAndUsage`
`ce.get_cost_and_usage_with_resources`	`ce:GetCostAndUsageWithResources`
`ce.get_cost_forecast`	`ce:GetCostForecast`
`ce.get_cost_categories`	`ce:GetCostCategories`
`ce.get_anomalies`	`ce:GetAnomalies`
`ce.get_rightsizing_recommendation`	`ce:GetRightsizingRecommendation`

`ce.get_rightsizing_recommendation`	`ce:GetRightsizingRecommendation`
`ce.get_savings_plans_coverage`	`ce:GetSavingsPlansCoverage`
`ce.get_savings_plans_purchase_recommendation`	`ce:GetSavingsPlansPurchaseRecommendatio`
`ce.get_reservation_recommendations`	`ce:GetReservationPurchaseRecommendation`
`budgets.describe_budgets`	`budgets:DescribeBudgets` + `budgets:ViewBudget`
`savingsplans.describe_savings_plans`	`savingsplans:DescribeSavingsPlans`
`savingsplans.describe_savings_plans_offerings`	`savingsplans:DescribeSavingsPlansOfferi`
Pricing MCP catalog reads	`pricing:GetProducts` / `DescribeServices` `GetAttributeValues`
`bcm-data-exports.list_exports`	`bcm-data-exports:ListExports`
`s3.list_objects_v2` / `get_object` (FOCUS bucket)	`s3:ListBucket` / `s3:GetObject` on `digitomics-focus-export-*`
`ec2.describe_regions`	`ec2:DescribeRegions`
`ec2.describe_instances`	`ec2:DescribeInstances`
`ec2.describe_volumes`	`ec2:DescribeVolumes`
`ec2.describe_addresses`	`ec2:DescribeAddresses`
`ec2.describe_snapshots`	`ec2:DescribeSnapshots`
`ec2.describe_vpcs`	`ec2:DescribeVpcs`
`ec2.describe_nat_gateways`	`ec2:DescribeNatGateways`
`ec2.describe_reserved_instances`	`ec2:DescribeReservedInstances`
`ec2.describe_images`	`ec2:DescribeImages`
`elbv2.describe_load_balancers`	`elasticloadbalancing:DescribeLoadBalanc`
`rds.describe_db_instances`	`rds:DescribeDBInstances`
`rds.describe_reserved_db_instances`	`rds:DescribeReservedDBInstances`
`elasticache.describe_cache_clusters`	`elasticache:DescribeCacheClusters`
`redshift.describe_clusters`	`redshift:DescribeClusters`

`redshift.describe_clusters`	`redshift:DescribeClusters`
`dynamodb.list_tables`	`dynamodb:ListTables`
`ecs.list_clusters` / `list_services` / `list_tasks`	`ecs:ListClusters` / `ListServices` / `ListTasks`
`eks.list_clusters`	`eks:ListClusters`
`lambda.list_functions`	`lambda:ListFunctions`
`s3.list_buckets`	`s3:ListAllMyBuckets`
`s3.get_bucket_location`	`s3:GetBucketLocation`
`cloudfront.list_distributions`	`cloudfront:ListDistributions`
`cloudwatch.get_metric_statistics`	`cloudwatch:GetMetricStatistics`
`cloudwatch.describe_alarms`	`cloudwatch:DescribeAlarms`
`iam.list_users`	`iam:ListUsers`
`ssm.get_parameter` (`/aws/service/*`)	`ssm:GetParameter`
`cloudtrail.lookup_events`	`cloudtrail:LookupEvents`
`s3.create_bucket` / `put_public_access_block` / `put_bucket_encryption` / `put_bucket_policy`	`s3:CreateBucket` / `s3:PutBucketPublicAccessBlock` / `s3:PutEncryptionConfiguration` / `s3:PutBucketPolicy`
`bcm-data-exports.create_export`	`bcm-data-exports:CreateExport`
`savingsplans.create_savings_plan`	`savingsplans:CreateSavingsPlan`

Full boto3 client list used anywhere in the backend: `sts`, `ce`, `budgets`, `ec2`, `rds`, `ecs`, `eks`, `lambda`, `s3`, `elbv2`, `cloudwatch`, `iam`, `cloudfront`, `elasticache`, `dynamodb`, `redshift`, `savingsplans`, `bcm-data-exports`, `cloudtrail` + `pricing` (via the Pricing MCP subprocess) + `ssm` (AMI lookup). No `cur`, `account`, `invoicing`, `billing`, `payments`, `tax`, `billingconductor`, `freetier`, `organizations` (writes), `compute-optimizer`, `cost-optimization-hub`, or `support` client is ever constructed.

---

6. What `AWSBillingReadOnlyAccess` grants that policy #1 deliberately drops

`AWSBillingReadOnlyAccess` (current managed version) grants, roughly: `account:GetAccountInformation`, `billing:Get*` / `List*`, `ce:*` (read)

`account:GetAccountInformation`, `billing:Get*` / `List*`, `ce:*` (read), `consolidatedbilling:Get*` / `List*`, `cur:Get*` / `Validate*`, `freetier:Get*`, `invoicing:Get*` / `List*`, `payments:Get*` / `List*`, `tax:Get*` / `List*`, plus (in older revisions) `aws-portal:View*`, `budgets:ViewBudget`, `pricing:*`, `purchase-orders:ViewPurchaseOrders`.

Policy #1 keeps only `ce:*` (read), `budgets:*` (read), `savingsplans:Describe*`, `pricing:*` (read), `bcm-data-exports:*` (read), `sts:GetCallerIdentity`, and the scoped FOCUS bucket read. Everything else in the managed policy — `invoicing:*`, `tax:*`, `payments:*`, `freetier:*`, `account:*`, `cur:*`, `consolidatedbilling:*`, `purchase-orders:*`, console `billing:*` — is **never called by any Digitomics code path** and is dropped.

Trade-off: with policy #1 instead of the managed policy, this IAM principal can no longer open the AWS **console** "Bills / Payments / Tax / Purchase orders" pages. The Digitomics product does not use them. Keep `AWSBillingReadOnlyAccess` *instead of* #1 only if a human also needs those console pages under this same principal.

---

7. Attaching

Console

1. **IAM → Policies → Create policy → JSON** — paste #1, name it `DigitomicsAwsConnectBillingRead`. Repeat for #2, and any of #3–#6 you need.
2. **IAM → Users** → your `digitomics-connector` user → **Add permissions → Attach policies directly** — select #1 + #2 (+ optionals). **Do not** select `ReadOnlyAccess` or `AWSBillingReadOnlyAccess`.
3. Rotate/create the access key and paste it into **Dashboard → Data Sources → AWS**.

CLI

```
for p in DigitomicsAwsConnectBillingRead DigitomicsAwsConnectInventoryRead; do
  aws iam create-policy --policy-name "$p" --policy-document "file://$p.json"
  aws iam attach-user-policy --user-name digitomics-connector \
    --policy-arn "arn:aws:iam::<ACCOUNT_ID>:policy/$p"
done

# remove the managed policies if previously attached
aws iam detach-user-policy --user-name digitomics-connector \
  --policy-arn arn:aws:iam::aws:policy/ReadOnlyAccess
aws iam detach-user-policy --user-name digitomics-connector \
  --policy-arn arn:aws:iam::aws:policy/AWSBillingReadOnlyAccess
```

CloudFormation cross-account role

In `infra/aws/digitomics-aws-connect.yaml`, drop both

`arn:aws:iam::aws:policy/ReadOnlyAccess` and `arn:aws:iam::aws:policy/AWSBillingReadOnlyAccess` from `DigitomicsConnectRole.ManagedPolicyArns`, and add policies #1 and #2 (plus any optionals) as inline `Policies:` documents. Combined they are ~4 KB — within the 10,240-char inline limit.

---

8. Troubleshooting

Run **Dashboard → AWS → Refresh data & forecast** and watch the `errors[]` array (or backend logs: `action=aws_dashboard_task_failed`).

Symptom	Missing
Connect wizard red banner, won't save	#1 `AccountIdentity` (or bad keys / billing IAM toggle off)
Sync fails, "AWS Cost Explorer rejected this connection"	#1 `CostExplorerRead` **or** the account billing-IAM toggle
Budgets question in chat errors	#1 `BudgetsRead`
Commitments page empty for AWS	#1 `SavingsPlansRead` (Savings Plans) and/or #2 `ec2:Describe*` / `rds:Describe*` (Reserved Instances)
"estimate architecture cost" fails	#1 `PriceListRead`
FOCUS stuck on Cost-Explorer-only data	#1 `FocusDataExportRead` + `FocusExportBucketRead`; for auto-provisioning also #4
An inventory tile stuck at `0` + an `errors[]` entry naming that service	the matching `Sid` in #2
Utilization columns blank, no idle-resource findings	#2 `ObservabilityRead`
IAM tile `0`	#2 `IamAccountMetadataRead`
DevOps agent can't resolve an AMI	#2 `PublicSsmAmiParameterRead`
Anomaly cards have no "root cause"	#5
DevOps chat "list my `<service>`" → AccessDenied	add that service to #3

A valid key with a too-narrow policy still **saves** — the gap shows on the next refresh as an `errors[]` entry, not an STS rejection.

9. What these policies never grant

- No `s3:GetObject` / `s3:GetBucketPolicy` on arbitrary buckets — only the connector's own `digitomics-focus-export-*` buckets (#1 read, #4 write).
- No `dynamodb:GetItem` / `Query` / `Scan`, no `sqs:ReceiveMessage`, no `kms:Decrypt` / `GetKeyPolicy`, no `secretsmanager:GetSecretValue`, no `ssm:GetParameter` outside `/aws/service/*` — **zero data-plane read**.
- No `lambda:GetFunction*` (code download URL), no `ec2:GetPasswordData`, no `cloudfront:GetDistribution*` (custom-origin headers).
- No `iam:*` beyond three account-summary calls — nothing that reads a policy, key, password, or MFA device, and nothing that writes.
- No resource creation, modification, or deletion anywhere except the opt-in, name-scoped FOCUS export bucket/definition in #4.
- No `sts:AssumeRole` on the customer principal, no `organizations:*` writes, no account, billing-preference, or support-plan changes.
- No console `billing:*` / `invoicing:*` / `tax:*` / `payments:*` / `cur:*` — the connector reads spend only through Cost Explorer.